

1. Introduction

This chapter provides the background, problem statement, objectives, scope, and organization of the report.

1.1 Background/Problem Statement

Mobile gaming remains a popular form of entertainment, but many games require an internet connection, limiting accessibility. While some offline games exist, they often lack variety and engaging features. Many gaming apps fail to provide:

- A diverse collection of games in one platform,
- Seamless offline functionality,
- User-friendly and engaging interfaces, and
- A smooth, lag-free experience.

Fun Game was conceptualized to address these gaps and offer a comprehensive offline gaming experience, ensuring entertainment without internet dependency.

1.2 Objectives of the Project

The primary objectives of Fun Game are:

- To develop an engaging offline gaming platform with a variety of six games in one app.
- To provide a smooth and lag-free gaming experience without requiring an internet connection.
- To design a user-friendly interface that enhances accessibility and ease of use.
- To ensure optimized performance and reliability across different Android devices.
- To offer a diverse and immersive gaming experience that keeps users entertained.

1.3 Scope of the Project

The initial focus of Fun Game is to provide an engaging offline gaming experience for Android users, catering to a broad audience across different age groups. The app includes six unique games in one platform, ensuring entertainment without an internet connection.

1.4 Organization of the Report

This report is structured to detail the development of Fun Game through the following chapters:

- **Introduction:** Provides background, objectives, and the need for an offline gaming platform.
- **Literature Survey:** Reviews similar gaming applications and identifies key gaps in existing solutions.
- **System Analysis:** Details system requirements, feasibility study, and the technologies used.
- **Design:** Covers the app's architecture, UI/UX design, and game mechanics.
- **Implementation:** Explains the coding approach, key game modules, and functionality.
- **Testing:** Describes testing methods, including performance and user experience testing.
- **Results and Discussion:** Highlights key outcomes, user feedback, and challenges encountered.
- **Conclusion:** Summarizes the project's achievements and suggests future improvements, such as adding new games or features.

2. Literature Survey/Review

2.1 Overview of Existing Work

Existing mobile gaming applications serve a vast audience, but many offline games fail to provide a diverse and engaging experience in a single platform. Common limitations of existing offline games include:

- Limited variety, requiring users to install multiple apps for different games.
- Lack of engaging mechanics, reducing long-term user interest.
- Outdated user interfaces that impact ease of use and accessibility.

2.2 Projects or Products Reviewed

Standalone Offline Games (Sudoku, Memory Games, etc.)

- Strengths: Simple gameplay, no internet dependency.
- Weaknesses: Each game is a separate app, requiring multiple installations.

Online Gaming Platforms

- Strengths: Offer multiplayer features and a wide variety of games.
- Weaknesses: Require internet access, limiting accessibility.

Hybrid Games (Both Online & Offline Modes)

- Strengths: Provide flexibility with online and offline play.
- Weaknesses: Often prioritize online features, leading to limited offline functionality.

2.3 Identification of Gaps

- Combines multiple games in one app, reducing the need for multiple installations.
- Provides an engaging and interactive experience with optimized game mechanics.

3. System Analysis

3.1 Requirements Specification

Functional Requirements:

- Multiple offline games integrated into a single platform.
- Intuitive game selection menu for easy navigation.
- Progress tracking for each game (e.g., scores, levels).
- Responsive and engaging UI for smooth gameplay.
- Optimized performance to ensure a lag-free experience on various Android devices.

Non-functional Requirements:

- Efficiency: Minimal load time and optimized resource usage.
- Compatibility: Support for different screen sizes and Android versions.
- User-friendly design: Simple and engaging interface for all age groups.
- Reliability: Ensure smooth and bug-free performance across all games.

3.2 Feasibility Study

Technical Feasibility:

- The project is developed in Android Java, ensuring compatibility with a wide range of devices.
- Uses Android's built-in UI components and game logic optimization for a smooth user experience.
- Does not require a backend server, making it lightweight and fully offline.

Economic Feasibility:

- Low development and maintenance costs, as the app runs without server dependencies.
- Revenue potential through in-app ads or future premium versions with additional games and features.
- No recurring costs for hosting or data storage since the app is completely offline.

Operational Feasibility:

- Simple and user-friendly interface, making it accessible for all age groups.
- No internet dependency, ensuring users can play anytime, anywhere.
- Optimized for low-power devices, ensuring smooth performance across different Android versions.

3.3 Tools and Technologies Used

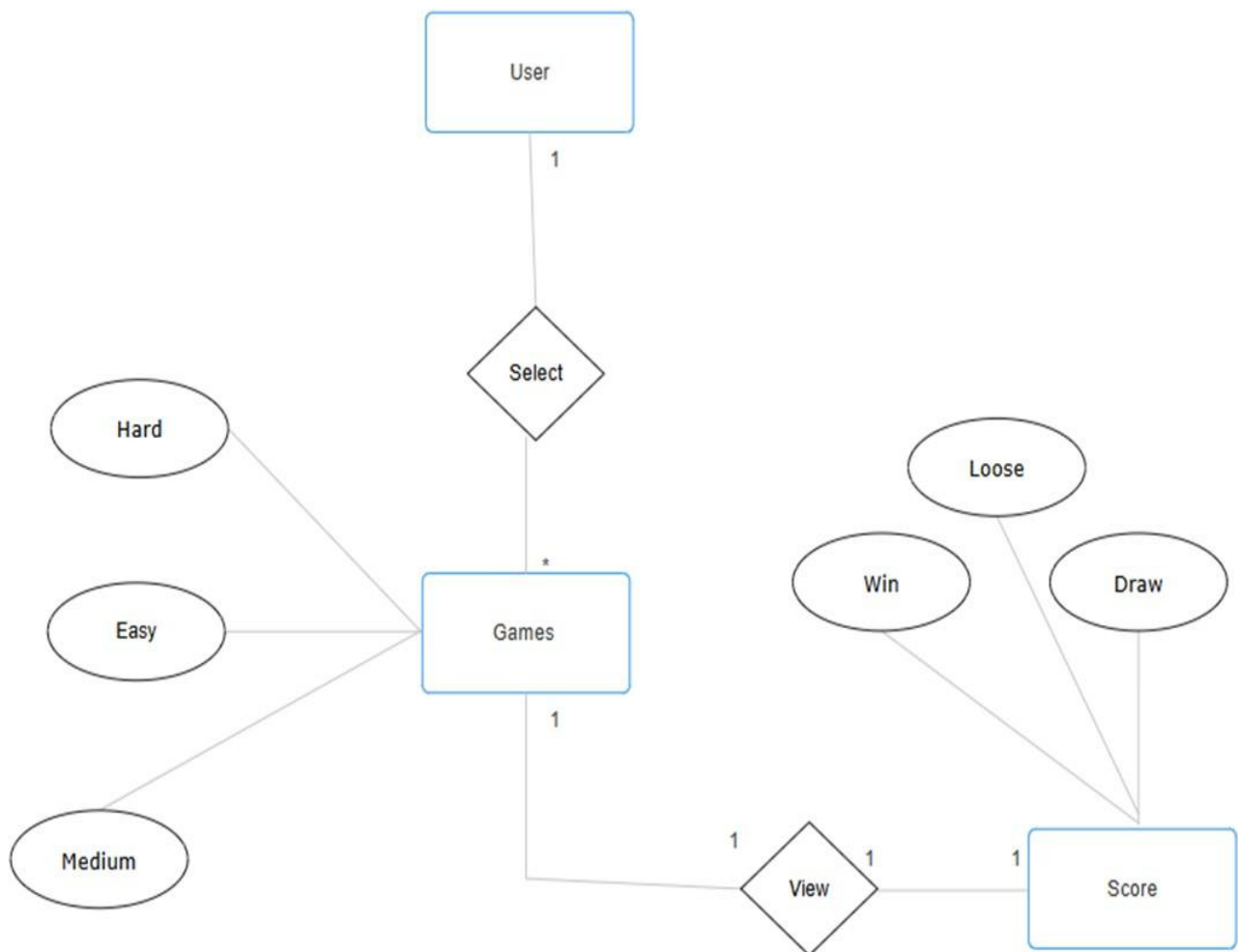
- Android Java – Core language for app development.
- Android SDK – Provides necessary tools and libraries for building the app.
- XML – Used for designing the user interface.
- Android Studio – Integrated development environment (IDE) for development and testing.

4. Design

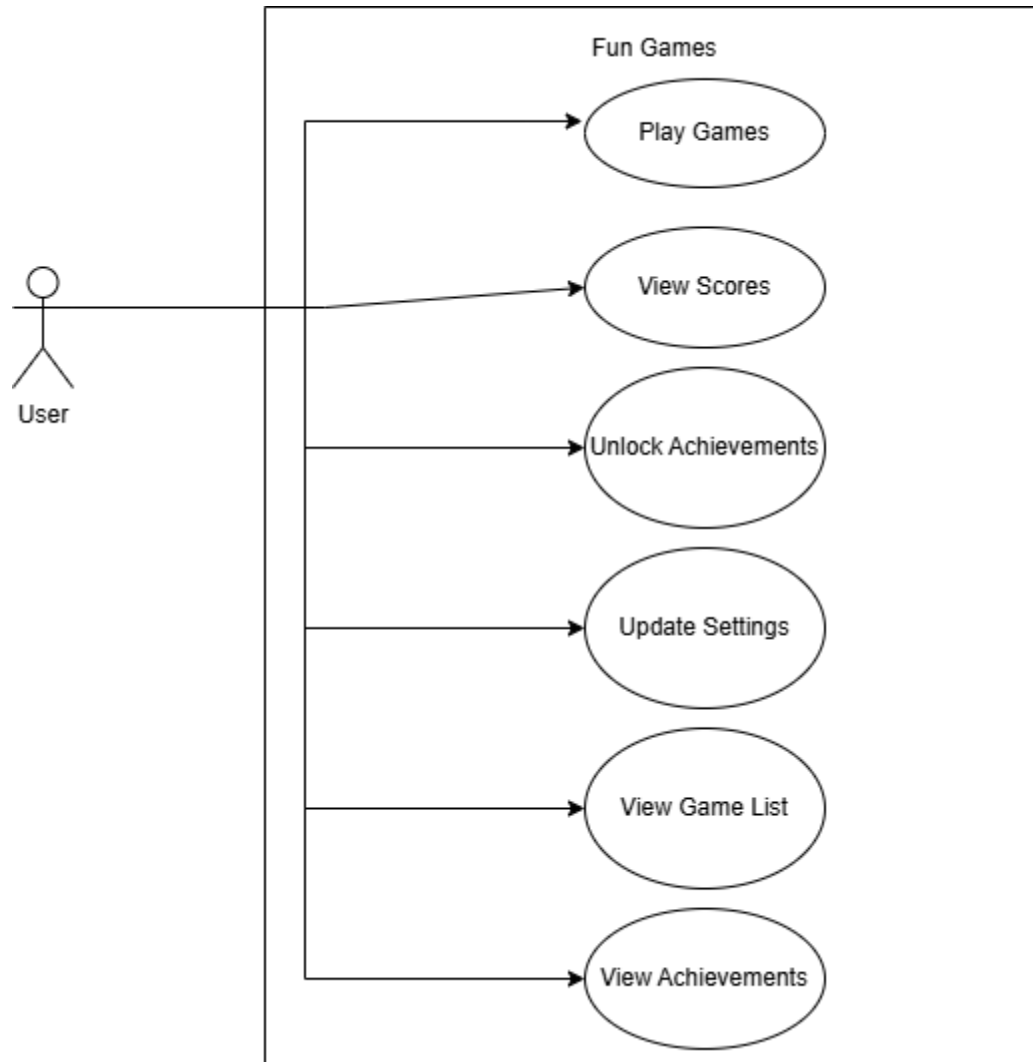
4.1 System Architecture

The architecture comprises:

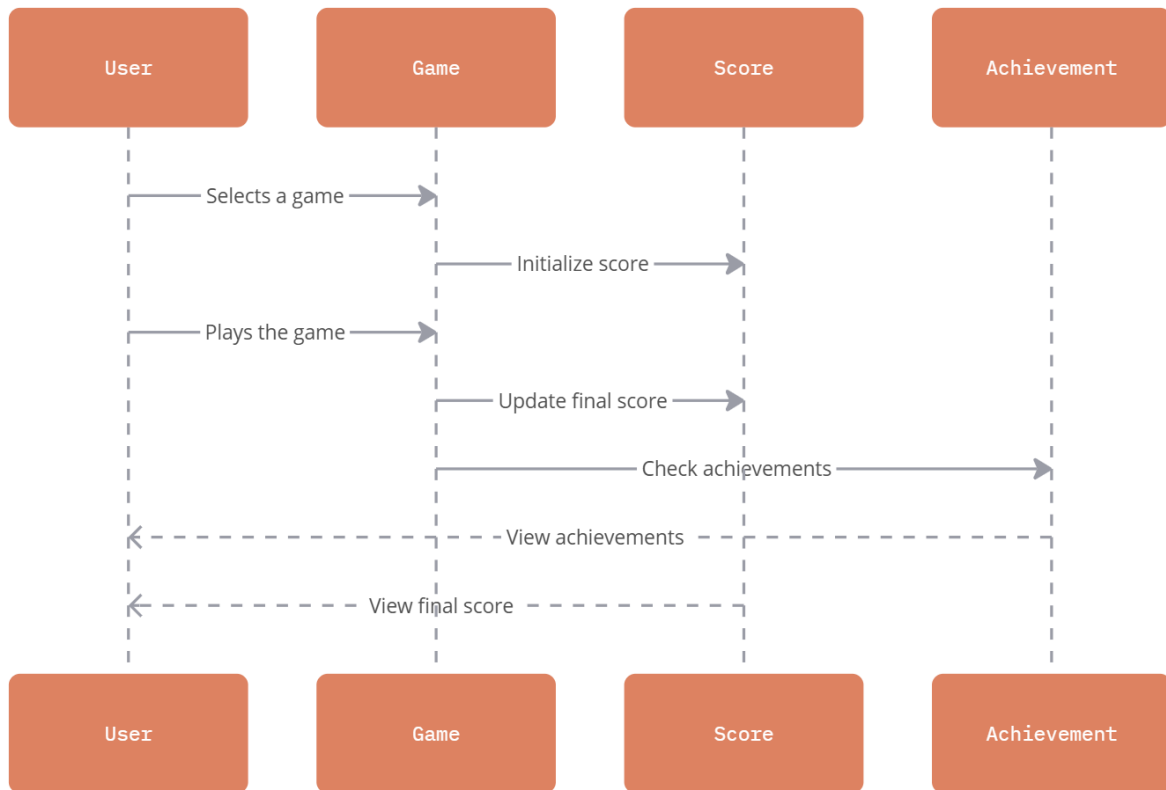
- User Interface Layer: Handles game selection and display.
- Application Layer: Manages game logic, scoring, and interactions.



4.2 UML Diagrams – Use Case Diagram

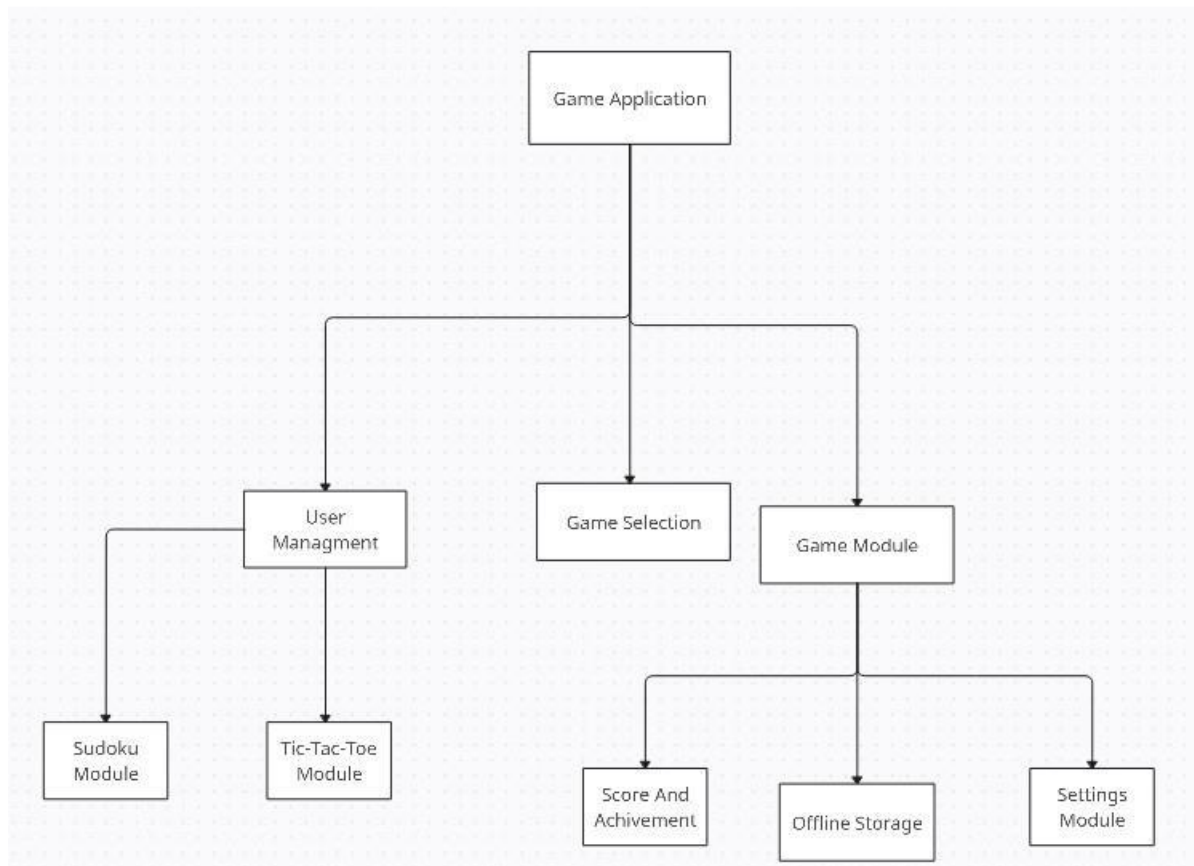


4.2 UML Diagrams – Sequence Diagram:

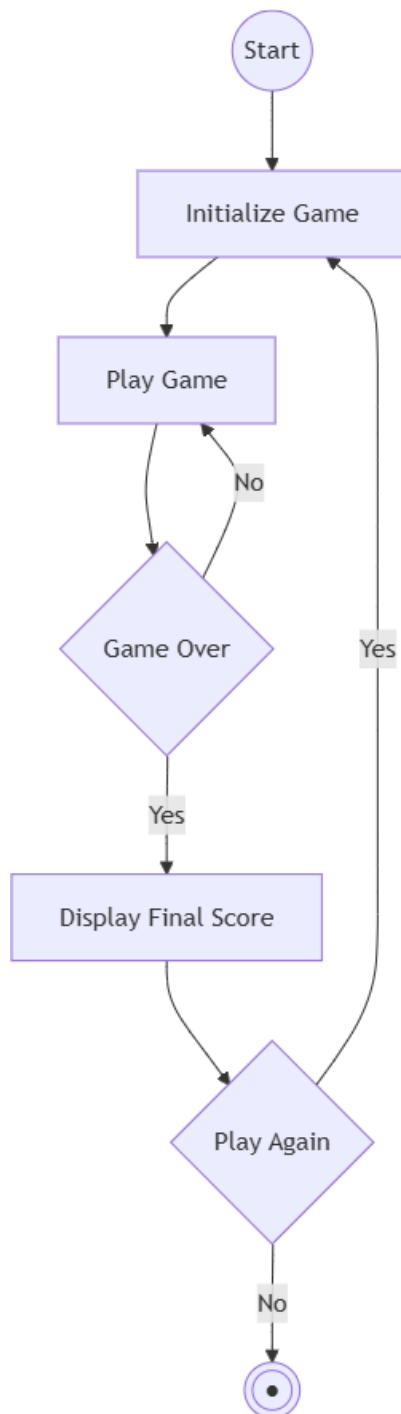


miro

4.3 UML Diagrams – Module Hierarchy:



4.4 UML Diagrams – Activity Diagram:



5. Implementation

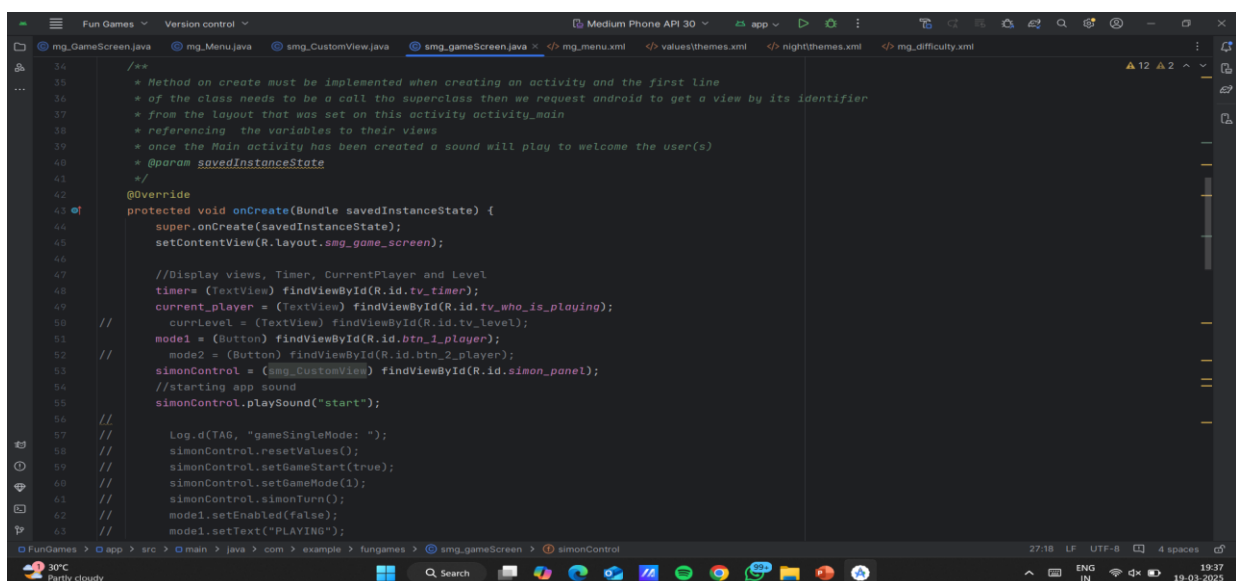
5.1 Description of Modules

The Fun Game app consists of the following key modules:

1. Game Selection Module:
 - Displays a menu for choosing from six offline games.
 - Ensures smooth navigation between games.
2. Game Logic Module:
 - Manages game rules, scoring, and difficulty levels.
 - Handles user interactions and animations.
3. Progress Tracking Module:
 - Saves scores in device storage(no database required).
 - Allows users to resume games where they left off.
4. User Interface Module:
 - Provides a responsive and user-friendly design.
 - Optimized for different screen sizes and resolutions.

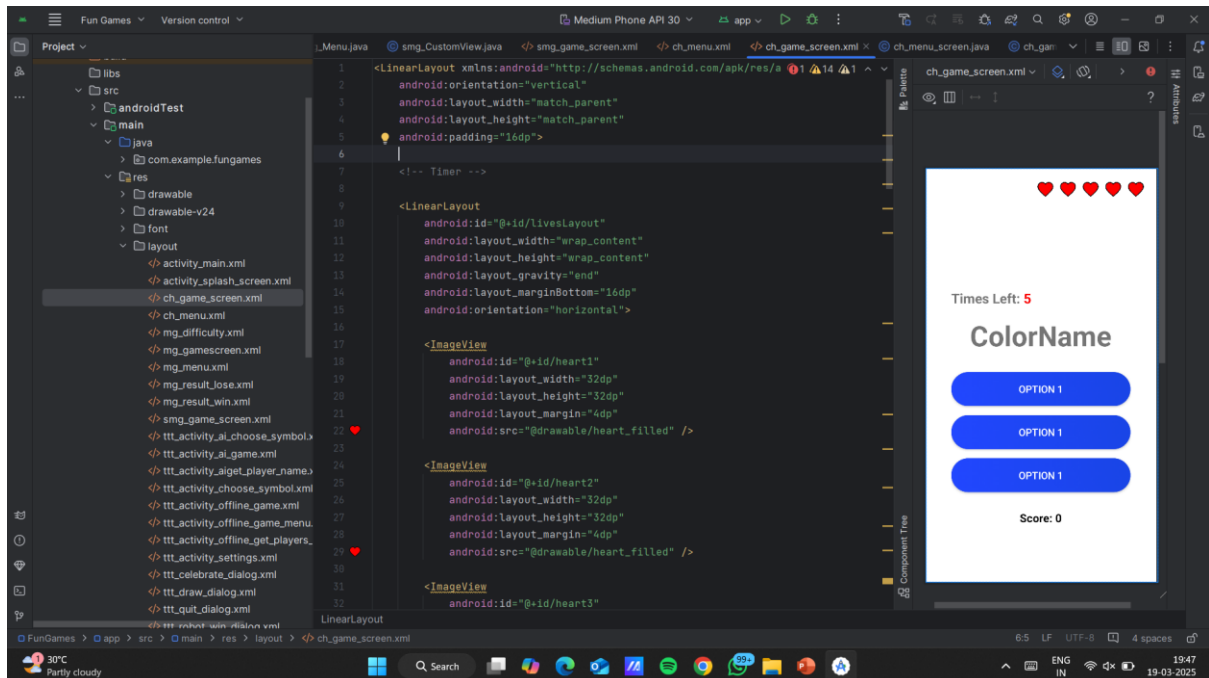
5.2 Code Snippets

1. Flappy Bird

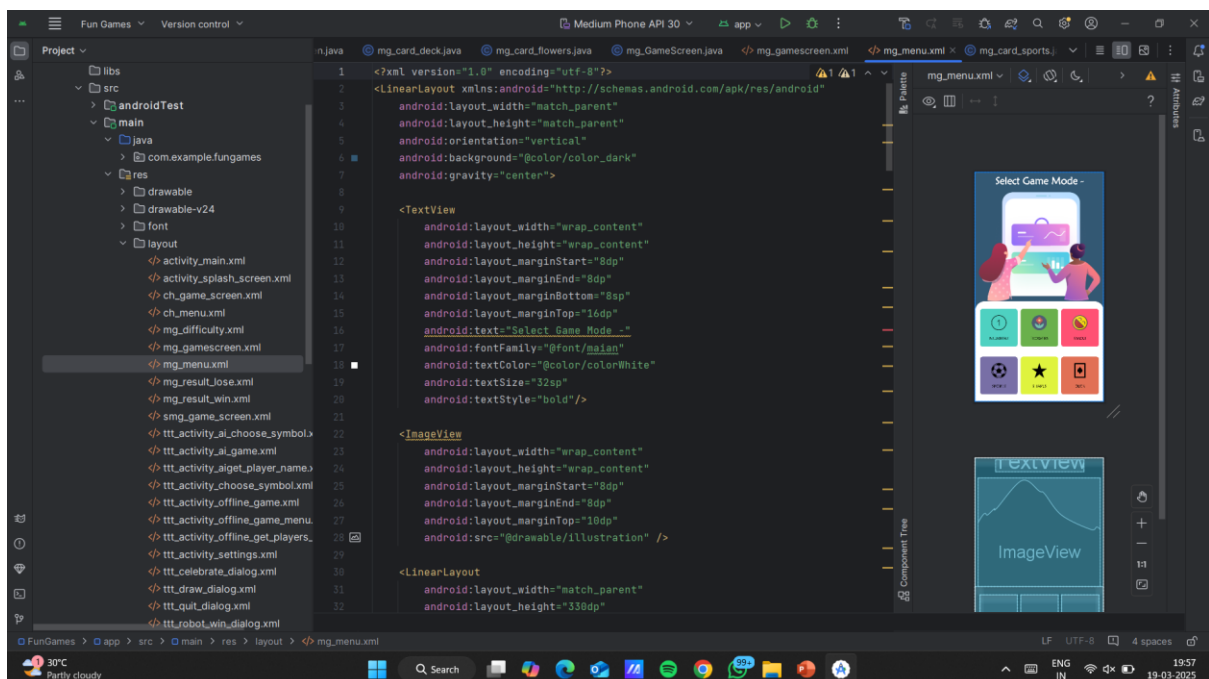


```
34  /**
35   * Method on create must be implemented when creating an activity and the first line
36   * of the class needs to be a call to the superclass then we request android to get a view by its identifier
37   * from the layout that was set on this activity activity_main
38   * referencing the variables to their views
39   * once the Main activity has been created a sound will play to welcome the user(s)
40   * @param savedInstanceState
41   */
42  @Override
43  protected void onCreate(Bundle savedInstanceState) {
44      super.onCreate(savedInstanceState);
45      setContentView(R.layout.smg_game_screen);
46
47      //Display views, Timer, CurrentPlayer and Level
48      timer = (TextView) findViewById(R.id.tv_timer);
49      current_player = (TextView) findViewById(R.id.tv_who_is_playing);
50      currLevel = (TextView) findViewById(R.id.tv_level);
51      mode1 = (Button) findViewById(R.id.btn_1_player);
52      mode2 = (Button) findViewById(R.id.btn_2_player);
53      simonControl = (Smg_CustomView) findViewById(R.id.simon_panel);
54      //starting app sound
55      simonControl.playSound("start");
56
57      Log.d(TAG, "gameSingleMode: ");
58      simonControl.resetValues();
59      simonControl.setGameStart(true);
60      simonControl.setGameMode(1);
61      simonControl.simonTurn();
62      mode1.setEnabled(false);
63      mode1.setText("PLAYING");
64  }
```

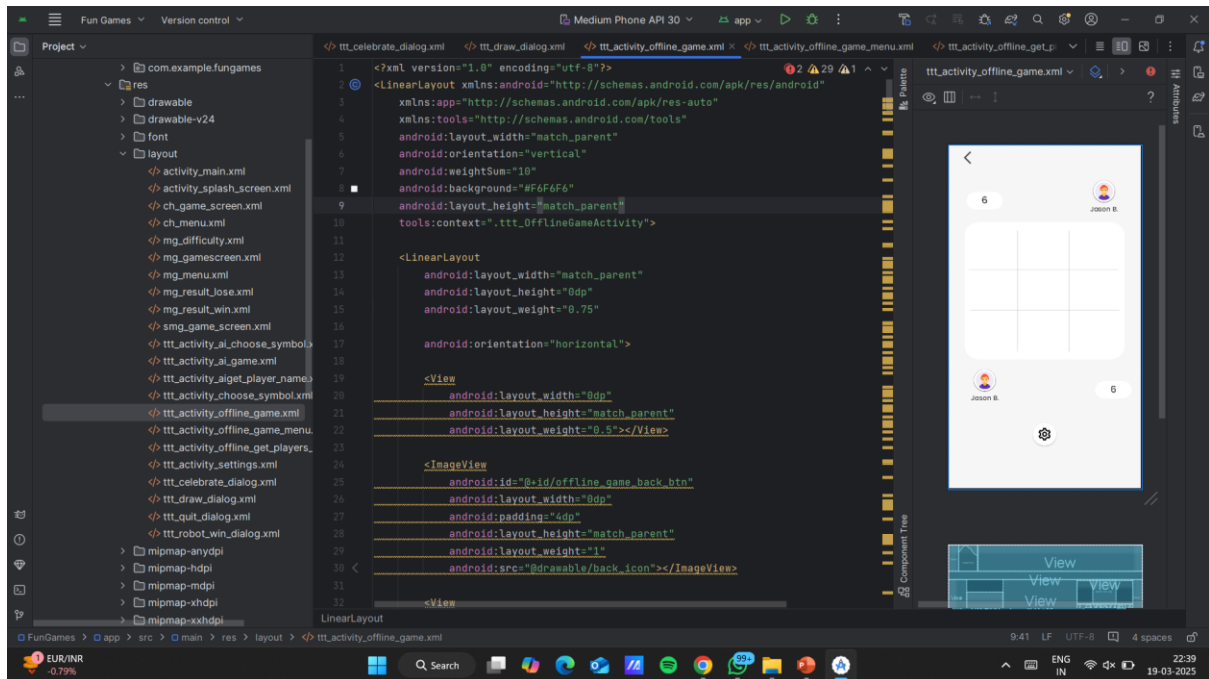
2.Colour Hunt



3.SoundMemory



4. TicTacToe

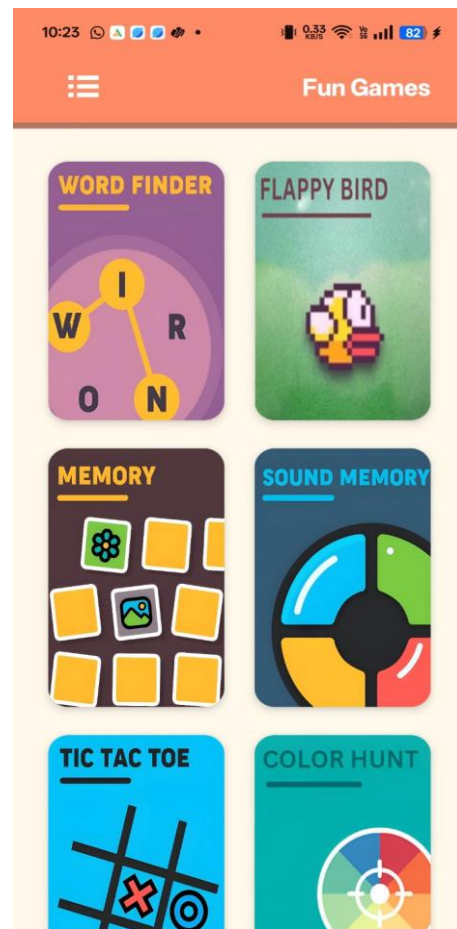


5.3 Screenshots of the Website

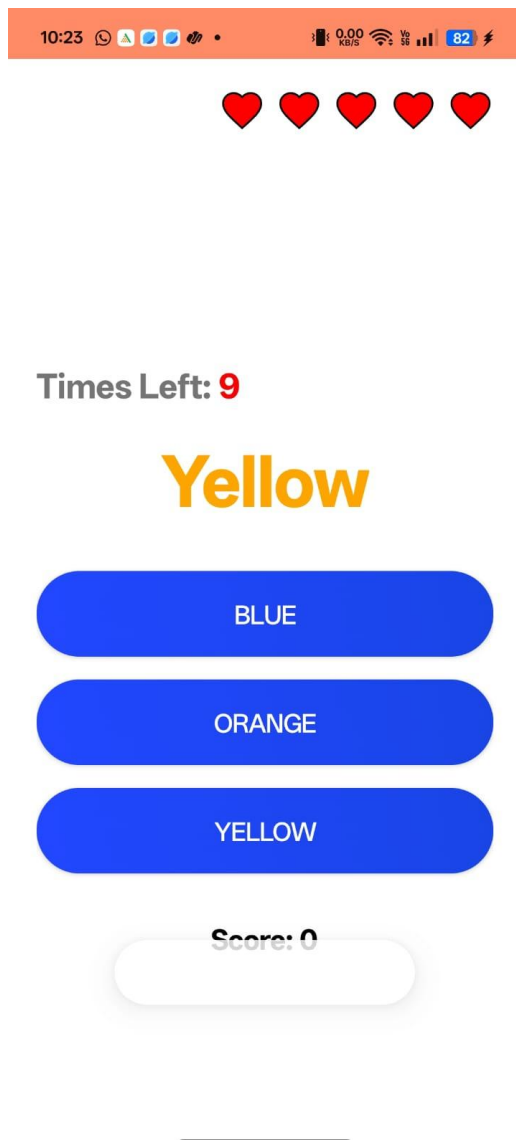
1. Splash :



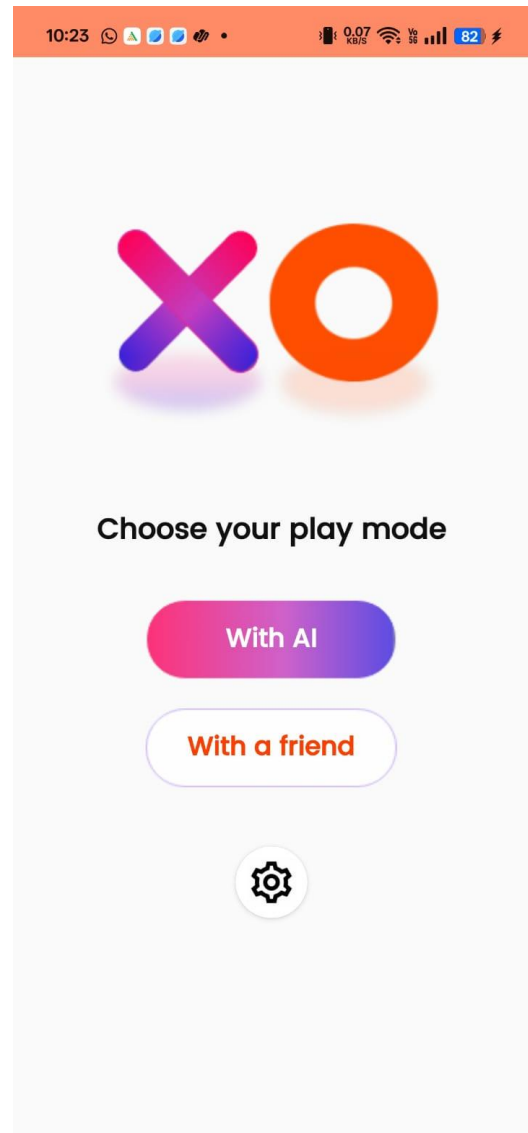
2. Dashboard Page:



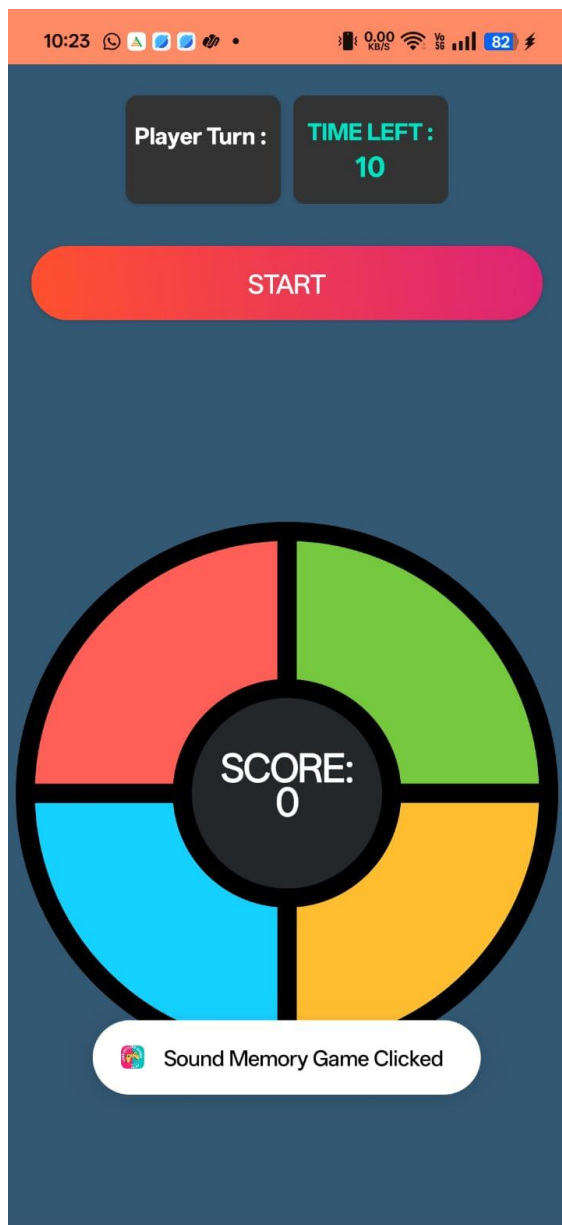
2. Color Hunt:



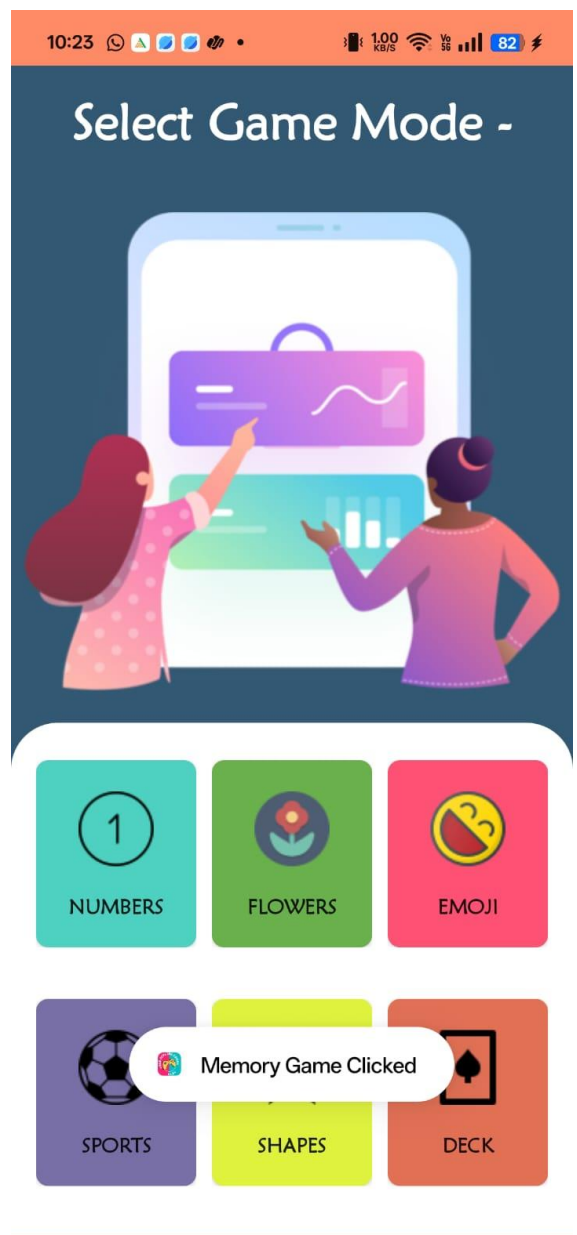
3. Tic- Tac-Toe



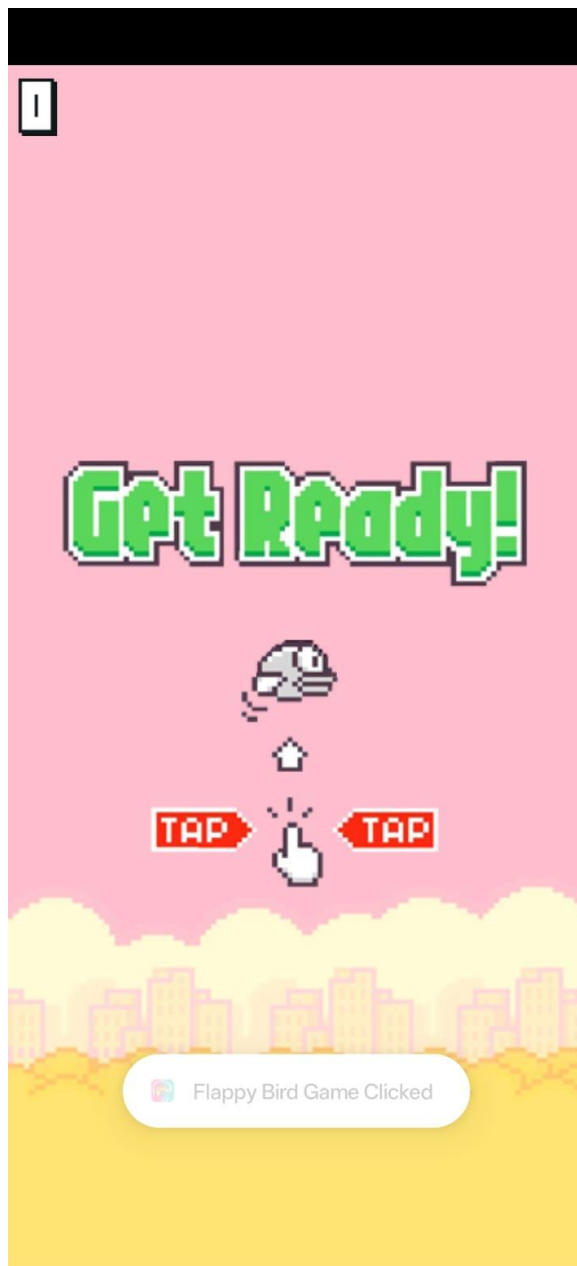
3. Sound Memory:



4. Card Memory:



4. Flappy Bird:



5. Word Game:



6. Testing

6.1 Test Cases and Testing Methodology

Project Name: Fun Game	Module Name: Game Selection, Game Logic, Progress Tracking	Project Code: FunGame
Total No. of Test Cases: 6	Total Passed: 6	Total Failed: 0
Total Executed: 6		

Game Selection Module

Test Case ID	Test Case Procedure	Input Data	Expected Output	Actual Output	Test Status
1	Check functionality of game selection.	Select a game from the menu.	Selected game should launch successfully.	Selected game launches successfully.	Pass
2	Check functionality when no game is selected.	Do not select any game and wait.	No action should occur.	No action occurs.	Pass

Game Logic Module

Test Case ID	Test Case Procedure	Input Data	Expected Output	Actual Output	Test Status
1	Check if scoring system works correctly.	Play and earn points.	Score should update correctly.	Score updates correctly.	Pass
2	Check game restart functionality.	Restart the game after losing.	Game should reset properly.	Game resets properly.	Pass

Progress Tracking Module

Test Case ID	Test Case Procedure	Input Data	Expected Output	Actual Output	Test Status
1	Check if progress is saved.	Play a game and exit.	Progress should be saved.	Progress is saved using SharedPreferences.	Pass
2	Check if progress resets on game restart.	Restart the game.	Progress should reset to default.	Progress resets to default.	Pass

6.2 Testing Methodology

1. Unit Testing: Focused on individual modules like game selection, game logic, and progress tracking.
2. Integration Testing: Verified seamless interaction between game selection, scoring, and UI elements.
3. User Acceptance Testing (UAT): Gathered feedback from users to enhance gameplay experience and UI design.

6.3 Results of Testing

All core functionalities, including game selection, scoring, and progress tracking, were tested under various scenarios. The app demonstrated stability, responsiveness, and smooth performance, with minor UI adjustments made for better user experience.

7. Results and Discussion

7.1 Outcomes of the Project

The Fun Game app successfully achieved the following objectives:

1. Multiple Games in One App: Users can enjoy six different offline games on a single platform.
2. Smooth Gameplay Experience: Optimized performance ensures lag-free interactions.
3. User-Friendly Interface: Intuitive design makes game selection and navigation effortless.
4. Offline Accessibility: No internet required, allowing users to play anytime, anywhere.

7.2 Comparison with Existing Solutions

Feature	Fun Game	Standalone Offline Games
Multiple Games in One App	Yes	No (Single game per app)
Offline Accessibility	Fully offline	Yes
User-Friendly Interface	Optimized UI	Basic UI
Progress Tracking	No tracking	No tracking

7.3 Challenges Faced

1. UI Optimization: Ensuring smooth performance across different screen sizes required extensive testing.
2. Game Logic Implementation: Balancing difficulty levels and ensuring fair gameplay took multiple iterations.

8. Conclusion and Future Scope

8.1 Summary of the Work

The Fun Game app successfully delivers an engaging collection of six offline games in a single platform. By focusing on user-friendly design, smooth performance, and offline accessibility, the app provides a seamless gaming experience.

8.2 Limitations

1. Limited Game Collection: Currently includes six games, but more can be added.
2. No Online Features: Lacks multiplayer mode or leaderboard integration.

8.3 Future Enhancements

1. New Game Additions: Expand the app with more interactive games.
2. Difficulty Customization: Allow users to adjust difficulty levels for a personalized experience.
3. Multiplayer Mode: Introduce local multiplayer support for shared gameplay.
4. Achievements & Rewards: Implement a reward system to boost engagement.

9. References

- Android Developer Documentation.

<https://developer.android.com>

- Java Official Documentation.

<https://docs.oracle.com/en/java>

- Offline Games

<https://playstore.com>

- Game Development Best Practices.

<https://gamedevelopment.tutsplus.com>